

QA Engineer

Take-Home Technical Assessment

Mid- to Senior-Level • 2-Day Window • ~4–6 Hour Effort

1. Overview

Welcome, and thanks for your interest in joining the team. This exercise is designed to give us a realistic look at how you approach test design, exploratory testing, automation, and communication — the day-to-day work of a QA engineer here.

You'll be testing a real, live, production web form. We are not looking for exhaustive coverage; we are looking for thoughtful prioritization, clean code, and clear reasoning. We'd rather see 12 sharp test cases than 50 shallow ones.

Time expectations

- **Total effort:** no more than 4–6 focused hours. Please don't burn a weekend on this.
- **Calendar window:** 2 business days from receipt.
- **Deliverable:** a public Git repo (or zip) plus a 15-minute walkthrough with our team.

2. System Under Test

URL: <https://public-site.stage.exclusiveresorts.com/inquire/>

This is the live public membership-inquiry form for Exclusive Resorts, a luxury vacation club. It's the top of their sales funnel — every lead flows into their CRM, so quality matters commercially.

Important — be a responsible tester. This is production. When submitting real form posts during exploratory testing, use clearly fake but recognizable data (e.g. name 'QA Candidate – [your name]', email like 'qa.candidate+test@yourdomain.com'). Do not flood the form. A handful of submissions across the assignment is plenty. For your automated tests, stub the network call (see §3.2) so CI runs don't create real CRM leads.

Form fields

- **Name** — required, free text
- **Email** — required, must be a valid email
- **Postal Code** — required, US and international
- **Phone** — required, numeric only, '+' allowed for international codes
- **Preferred Contact Method** — required, radio: Phone / Text / Email
- **Preferred Time of Day** — optional dropdown: Morning / Afternoon / Evening / I'm Flexible

- **Preferred Days** — optional dropdown: Monday-Friday / Saturday-Sunday / I'm Flexible
- **Privacy/Email Consent** — required checkbox
- **SMS Opt-in** — optional checkbox
- **Submit button**

3. What to Deliver

Five artifacts. Treat each as a real piece of work product — something you'd hand to a teammate.

3.1 Exploratory testing of the live form

Spend 45–60 minutes actually using the live form before you write any test cases or automation. Try to break it. Try edge cases. Watch DevTools (Network, Console) while you do.

Deliver a short bug report (Markdown or doc) listing every defect, UX issue, or suspicious behavior you find. We expect at least one finding — a sharp tester always finds something on a real form. If you genuinely find nothing, write a one-paragraph note explaining what you tried and why you concluded the form is solid.

For each bug, use this format:

ID	Title	Severity	Steps	Expected	Actual
BUG-01	(example) Phone field accepts letters when pasted	Medium	1. Paste 'abcd' into Phone 2. Tab out	Field rejects non-numeric input	Letters remain in field; form still submits

Tip: areas to poke at — paste behavior into Phone, very long inputs, unicode/emoji in Name, what happens on slow networks (DevTools throttling), browser back-button after submit, double-click on Submit, mobile viewport, screen-reader semantics.

3.2 Test plan & test cases

A short written test plan (Markdown or doc) covering:

- Scope and out-of-scope (what you're testing, what you're not, and why).
- Risk areas — where you think bugs are most likely to live, and how that drove your prioritization.
- A test-case table covering at least the categories below. Aim for ~12–15 cases.

Required categories

- Smoke: form loads and a happy-path submission works.
- Field validation: required fields, email format, phone numeric-only, postal code formats.
- Compliance: consent checkbox enforcement (this is a real legal requirement for marketing forms).
- Security: at minimum, an XSS check on a free-text field.
- UX / Accessibility: keyboard navigation, mobile viewport (375px), label associations.
- Edge cases: double-submit, very long input, international formats.

Sample test cases (yours can differ)

ID	Title	Type	Priority
TC-01	Page loads with all required fields visible	Smoke / UI	P0
TC-02	Submit valid lead — success message displayed	Smoke / E2E	P0
TC-03	Empty submit blocks and shows required-field errors	Negative	P0
TC-04	Malformed email rejected (e.g. 'foo@', 'foo.com')	Negative	P1
TC-05	Phone field rejects alphabetic characters	Negative	P1
TC-06	Consent checkbox required — submit blocked when unchecked	Compliance	P0
TC-07	XSS payload in Name renders as text, no script execution	Security	P0
TC-08	International phone with '+' prefix is accepted	Boundary	P2
TC-09	Postal code accepts US 5-digit and intl formats	Boundary	P2
TC-10	Radios (Phone/Text/Email) selectable & mutually exclusive	UI	P1
TC-11	Dropdowns expose only the documented options	UI	P2
TC-12	SMS opt-in is optional — form submits without it	Functional	P1
TC-13	Form is keyboard-navigable (tab order, Enter to submit)	Accessibility	P1
TC-14	Form is responsive at 375px (mobile) viewport	Responsive	P2
TC-15	Double-clicking Submit does not create duplicate leads	Concurrency	P1

3.3 Playwright UI automation

Pick at least 5 test cases from your plan and automate them in Playwright (TypeScript or JavaScript — your call). They must run against the live exclusiveresorts.com page.

What we'll be looking for:

- **Page Object Model** (or another clean abstraction). No raw selectors strewn across test bodies.
- **User-facing locators** (`getByLabel`, `getByRole`) over CSS/XPath where possible — they survive marketing's CSS refactors.
- **Network interception for the submit** — please do not fire real leads into their CRM on every CI run. Use `page.route()` to stub the POST and assert on the captured payload.
- **Test tagging** (`@smoke`, `@regression`, `@negative`) so we can run subsets.
- **Configuration**: `playwright.config` with retries, parallelism, HTML reporter, and at least 2 browsers (Chromium + WebKit is fine).

- **A CI workflow file** (GitHub Actions or similar). Doesn't need to be wired up to actually run — just committed.

Suggested layout

```
/playwright /pages/InquiryPage.ts /tests/inquiry-form.spec.ts
/fixtures/inquiry.fixtures.ts
playwright.config.ts.github/workflows/playwright.ymlREADME.md
```

3.4 Postman API automation

The form posts to a backend endpoint. Open DevTools → Network, submit the form once with valid data, and capture the request. Then build a Postman collection that exercises that endpoint directly.

Include the captured request as the baseline (200 case), then add negative and edge variants. Use Postman test scripts (pm.test) to assert on status, response time, and JSON shape. Parameterize with a Postman environment so the collection runs against staging vs production by switching one variable.

Deliver:

- collection.json (exported from Postman).
- environment.json (with placeholder values, no secrets).
- **A Newman command** in the README showing how to run it from CLI: `newman run collection.json -e environment.json`

Suggested API scenarios:

ID	Scenario	Expected
API-01	POST with all valid fields	2xx, success payload
API-02	POST missing a required field	4xx with field-level error
API-03	POST with malformed email	4xx validation error
API-04	POST without consent flag	4xx — server-side enforcement
API-05	POST with SQL-injection payload in name	Sanitized; no DB error leaked
API-06	GET on the submit endpoint	405 Method Not Allowed
API-07	Response time p95 over 10 calls	All < 2000ms
API-08	Response shape matches contract	Required keys present, correct types

Note: if the endpoint is heavily protected (CSRF tokens, bot-detection, etc.) and you can't get a clean 200 from Postman against production, document what you tried, what you observed, and what you'd do in a real engagement (e.g. ask for a staging endpoint, or use Playwright's `APIRequest` context). Showing your reasoning matters more than forcing a green run.

3.5 README & 15-minute walkthrough

A README at the repo root covering:

- Setup instructions (npm install, browsers, Postman/Newman).

- How to run smoke vs full regression.
- How CI is wired up.
- Known limitations and what you'd do with another day.

Then a 15-minute walkthrough with our team where you demo the suite live, show one test failing intentionally (break a selector), and walk us through your most interesting bug find.

4. How We'll Evaluate

Roughly weighted as follows:

- **Test design & prioritization (25%)** — do your test cases reflect real risk? Did you think about compliance, security, accessibility, not just happy paths?
- **Exploratory findings (20%)** — quality of bugs found on the live form. Sharp findings beat long lists of nitpicks.
- **Playwright code quality (25%)** — abstractions, locator choices, stability, configurability.
- **Postman/API rigor (15%)** — meaningful assertions, parameterization, runnable from CLI.
- **Communication (15%)** — README clarity, bug reports, and how you present the work in the walkthrough.

5. Constraints & Notes

- Use any open-source tooling you like. We're naming Playwright and Postman because we use them, but if you want to add Lighthouse, axe, or k6 alongside, go for it.
- AI assistance is fine and expected. Just be ready to explain every line in the walkthrough — we'll ask.
- If you hit a blocker (rate limiting, bot detection, captcha), document it and move on. Don't burn hours fighting infrastructure.
- Submit a public Git repo URL, or a zip via email, before the deadline.

6. Questions?

Email the hiring manager. We'd rather you ask one clarifying question than guess wrong for two hours. Good luck — we're looking forward to seeing your work.